

Humanoid Virtual Athletics Challenge SimuChalle 2026 Spring Meeting @Kawaruba

Choreonoid and vnoid

Introduction to Model-Based Control of Humanoid Robots

ver. 4/23 2026

Lecturer: Yuichi Tasaki (Kobe University)

self-introduction

- Career

History • 2016–Present: Associate Professor, Graduate School of Engineering, Kobe

University • 2009–Present: Assistant Professor, Graduate School of Engineering, Nagoya University

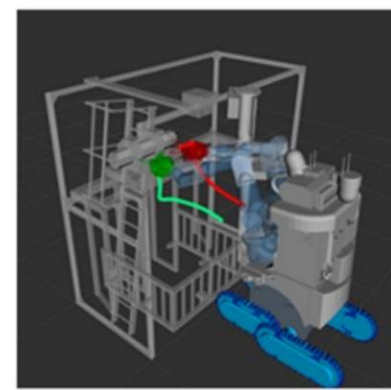
- Activities •

2021–Present: Chairperson of the Humanoid Virtual Athletics Challenge Executive Committee • 2017–Present: Member of the Humanoid Robotics Research Committee of the Robotics Society of Japan

- Research •

Motion planning, walking control, and whole-body motion control for humanoid robots • Map generation

and autonomous navigation for autonomous mobile robots • Remotely operated robots



This tutorial outline

- Part 0 (Preparation) Environment Setup

- Part 1: Kinematics •

- Theory •

- Practice •

Part 2: Center of Mass

Dynamics

- Theory • Practice

- Part 3: Gait Pattern Generation •

- Theory •

- Practice

environment construction

- ## What is Choreonoid?

- Integrated robot simulator by Shinichiro Nakaoka (<https://choreonoid.org/>)
- It mainly has simulation and robot choreography functions.
- Official simulator for the Simulation Challenge!

- ## What is vnoid?

- Developed by Tasaki as sample code for the simulation challenge
- [Humanoid robot control library \(https://github.com/ytazz/vnoid\)](https://github.com/ytazz/vnoid)
- The top priority is making it easy for beginners to read.

Its motto is simplicity and low functionality (but it's still reasonably walkable).

environment construction

The following describes the procedure for setting up a development environment using Windows + WSL.

Recommended environment (Disk space: approximately 4GB, Memory: 16GB or more recommended (8GB will result in slow builds)).

Set up WSL

Copying and pasting may cause character encoding issues and malfunctions , so please type it manually whenever possible!

Open Windows Terminal or Command Prompt.

Check the current installation status.

```
> wsl -l -v
```

If it displays as shown below, it is installed.

NAME	STATE	VERSION
Ubuntu	Stopped	2

To install, do the following:

```
> wsl --install -d Ubuntu
```

Set username and password as appropriate.

Start up

```
> wsl -d Ubuntu
```

This tutorial does not require you to pay particular attention to the Ubuntu version.

It should work similarly with any of the following versions: 20.04, 22.04, or 24.04.

environment construction

Please prepare a suitable text editor. The following instructions assume the use of Visual Studio Code.

Install Visual Studio Code

<https://code.visualstudio.com/>

environment construction

Choreonoid setup

For details, please refer to the official manuals: <https://choreonoid.org/ja/manuals/latest/install/build-ubuntu.html> <https://choreonoid.org/en/documents/latest/install/build-ubuntu.html>

Execute the following on the WSL terminal:

Navigate to the directory where you want to clone Choreonoid.
Generally, your home directory is recommended.

Please perform this process on WSL, not on the Git client on Windows.

```
> cd
```

```
> git clone https://github.com/choreonoid/choreonoid.git
```

```
> cd choreonoid
```

Install everything you need at once

```
> sudo apt update
```

```
> misc/script/install-requisites-ubuntu-24.04.sh
```

Make sure to match the Ubuntu version, otherwise it won't work properly.

Create a build directory and move to it.

```
> mkdir build &  
cd build
```

Execute Cmake

```
> cmake ..
```

Build. Please wait a moment.

```
> make -j8
```

environment construction

Choreonoid setup

For details, please refer to the official manuals: <https://choreonoid.org/ja/manuals/latest/install/build-ubuntu.html> <https://choreonoid.org/en/documents/latest/install/build-ubuntu.html>

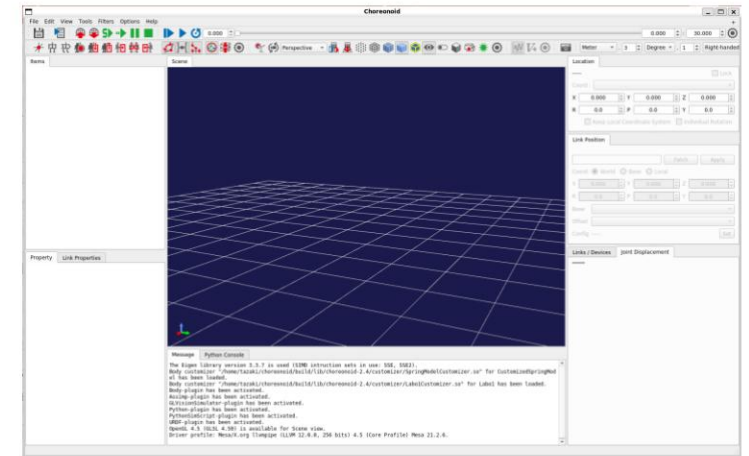
Try running it directly from the Build directory.

```
> bin/choreonoid
```

In an Ubuntu environment, you can run it directly from the build directory.

If you encounter a segmentation fault, add the following environment variables. *This may be resolved in future Choreonoid updates.

```
> export CNOID_USE_GLSL=0
```



If you run it after installing it

* While this is the correct method, it requires installation every time you build, so we will not use it in this tutorial.

Specify the installation location using CMake and run configure again.

```
> cmake .. -DCMAKE_INSTALL_PREFIX=/usr/local/choreonoid
```

Build & Install

```
> sudo make install
```

Run from the installation location.

```
> /usr/local/choreonoid/bin/choreonoid
```


environment construction

- Setting up the vnoid

Clone the vnoid under choreonoid/ext

```
> cd choreonoid/ext > git  
clone https://github.com/ytazz/vnoid.git
```

Source trees placed in the ext directory are automatically built together with choreonoid.

Run CMake again and build.

```
> cd ../build >  
cmake -DBUILD_VNOID_VISUALIZER_PLUGIN=1 .. > make  
-j8
```

Enable the option to build the visualizer plugin.

Running from the Build directory requires manually copying resource-related files (this is not necessary when using `make install`).

Copy the models used in this tutorial.

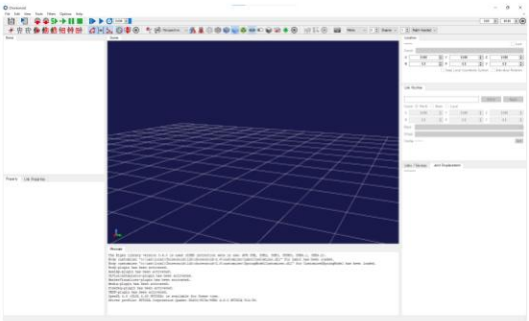
```
> cd choreonoid/ext/vnoid > cp -a  
model/* ../../build/share/choreonoid-2.4/model/
```

environment construction

- Setting up the vvoid

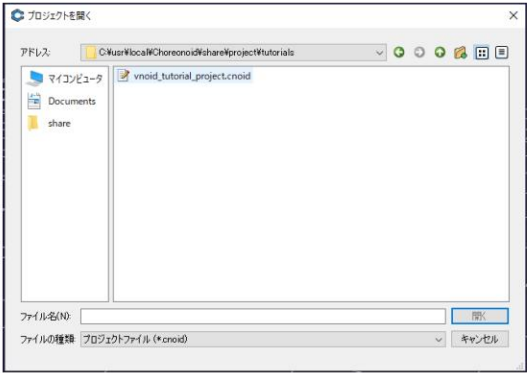
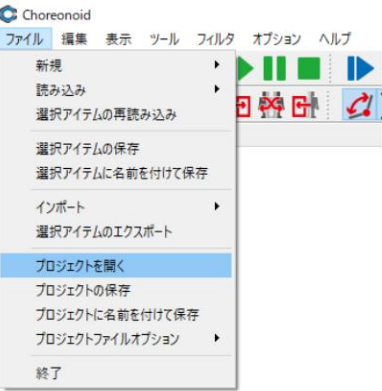
Let's try running the sample.

Run Choreonoid

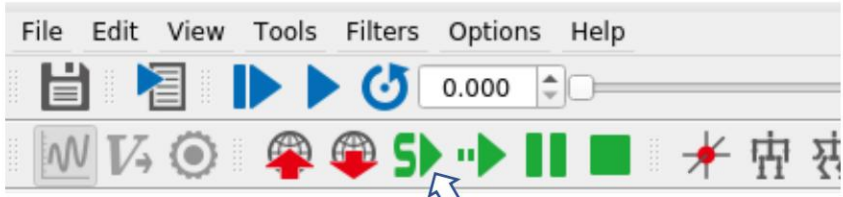


Open the following project file from the "Open Project" option in the "File" menu.

choreonoid/ext/vnoid/project/vnoid_sample_project_2023_athletics.cnoid

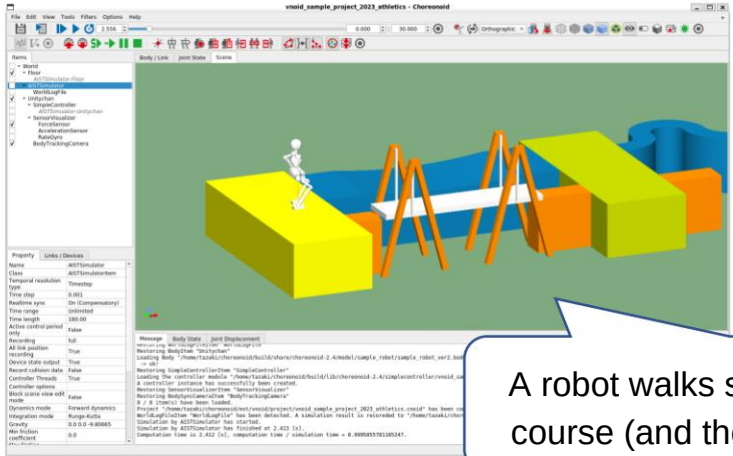


Run the simulation



This button

It's OK if it looks like this.



A robot walks straight on an athletic course (and then falls).

environment construction

- Setting up the vnoid

Load into Visual Studio Code

Launch VS Code and select the vnoid source tree from "Open Folder..." in the "File" menu.

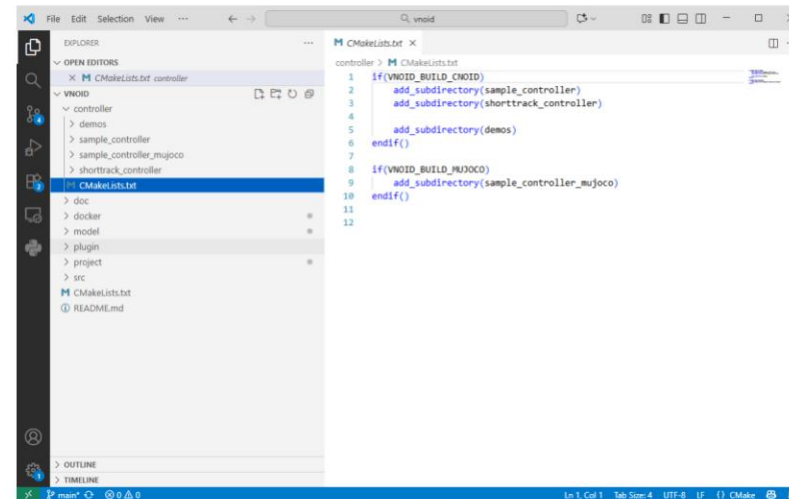
When opening a WSL folder from VS Code running on Windows, the path will be as follows:

If you clone choreonoid to /home/USERNAME:

\\wsl.localhost\Ubuntu\home\USERNAME\choreonoid\ext\vnoid

WSL Ubuntu image name

Username on Ubuntu



environment construction

- Setting up the vnoid

Now let's run this tutorial project. Run Choreonoid again

and open the following project file from "Open Project" in the "File" menu.

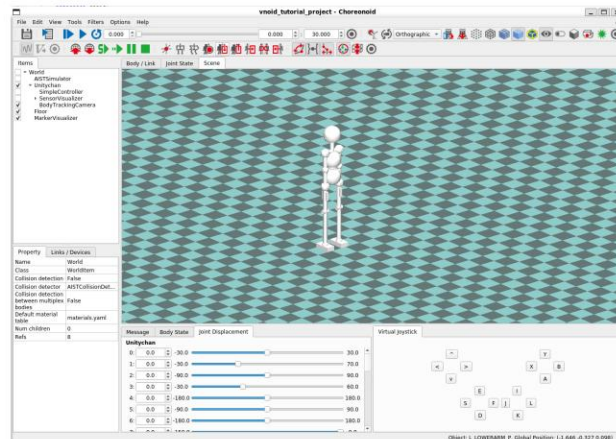
If you are running directly from the build directory

choreonoid/ext/vnoid/project/tutorials/tutorial1/vnoid_tutorial1_project.cnoid

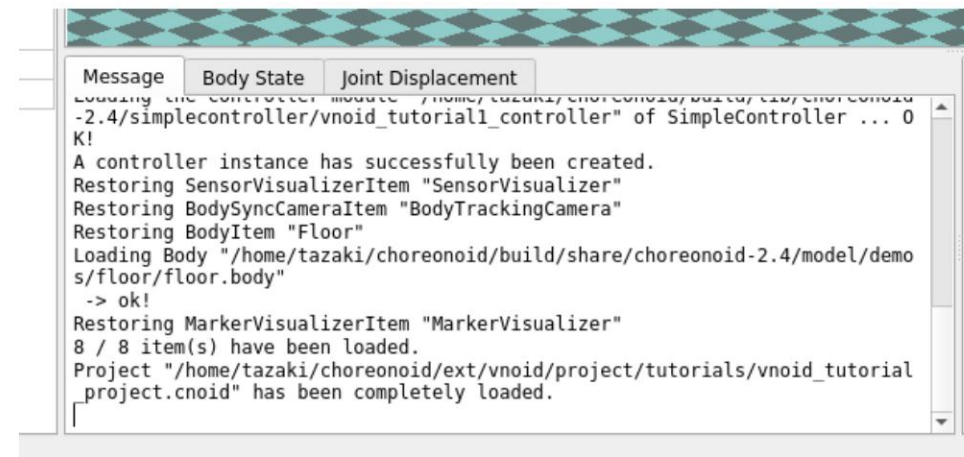
If you are running it after installation

[Choreonoid installation directory] /share/project/tutorials/tutorial1/vnoid_tutorial1_project.cnoid

It's OK if it looks like this.



Check the message view to see if there are any error messages (in red).



environment construction

Let's get used to the basic operations.

Starting and stopping the simulation

Items with a checkmark in the item tree are displayed in the scene view.

Camera rotation (left mouse button drag), Camera movement (middle mouse button drag), Zoom in/out (right mouse button drag)

You can change property simulation settings and other settings on the spot.

The robot's joint angles can be changed using the slider (these settings will be overwritten when the simulation is run).

Virtual joystick *Since WSL cannot recognize USB devices, we will utilize this.

kinematics

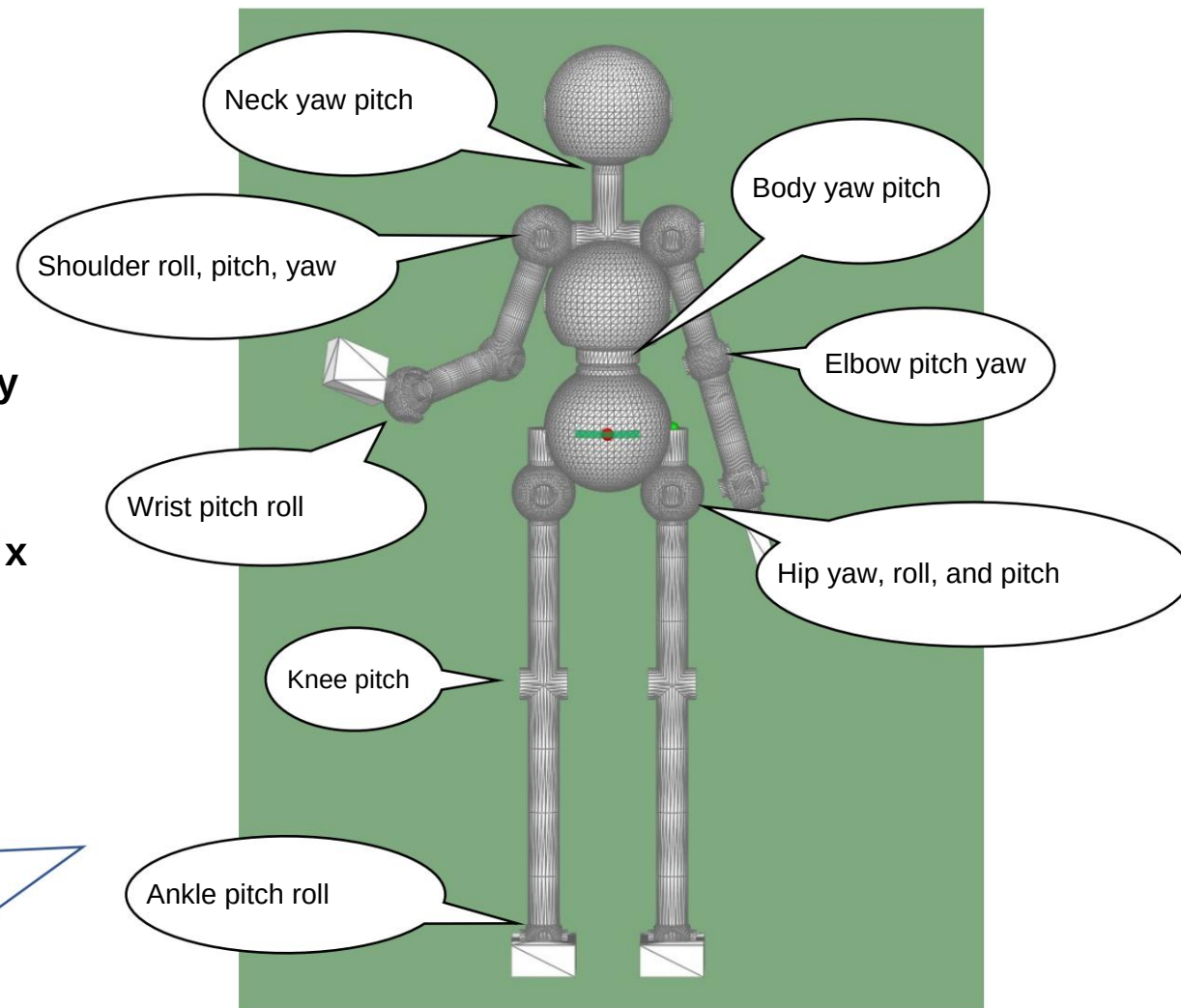
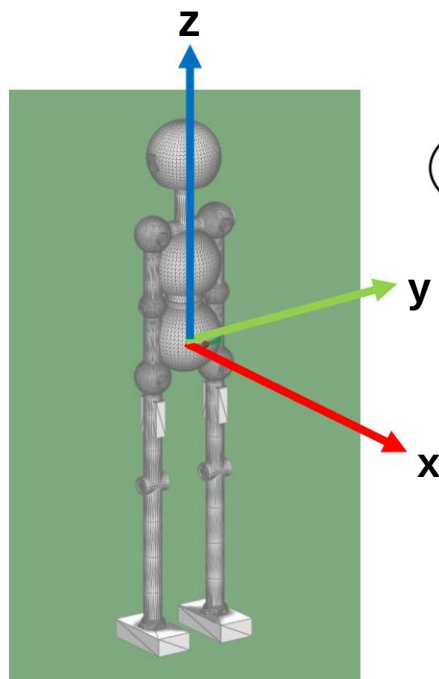
Kinematics

- Step 1: 30 minutes
 - Theory
 - Forward kinematics and inverse kinematics
 - Centroid inverse kinematics
 - Practical Application
 - Try running it with the base fixed in mid-air.
 - Try changing the joint angle using a gamepad.
 - Try moving your arms and legs using a gamepad.
 - Try releasing the base and lowering it to the ground.
 - Try making the character crouch using a gamepad.

kinematics

Regarding the terminology for axes

In the basic posture, the joint axes parallel to the x, y, and z axes will be called [joint location] [roll] pitch[yaw], respectively.



Be careful, as the way we refer to them might not be understood outside of this field (even within the humanoid community).

[Reference] It is incorrect to use the terms roll, pitch, and yaw to describe posture.

<https://qiita.com/zhidao/items/7ebdf70c4bbbed72243f9>

kinematics

Kinematics that often appear in humanoid robots

forward kinematics (FK)

The position and rotation of the limbs can be determined from the position and rotation of the base link and the angles of each joint.

Center of gravity forward kinematics

The position of the center of gravity can be determined from the position and rotation of the base link and the angles of each joint.

FK calculation is forward, so it's easy *in the case of serial links

inverse kinematics (IK)

The angles of each joint can be determined from the position and rotation of the base link and the position and rotation of the limbs.

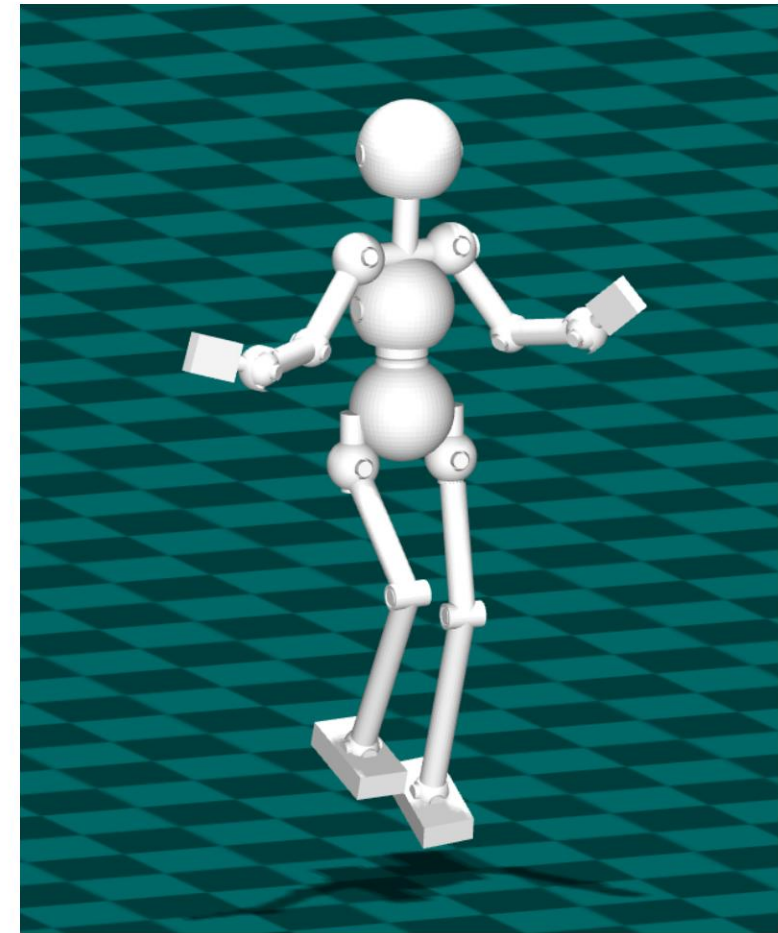
Center of gravity inverse kinematics

From the position of the center of gravity, the position and rotation of the limbs, we can determine the position and rotation of the base link and the angles of each joint.

IK is a bit difficult because it's the inverse problem.

Iterative solutions using differential inverse kinematics are well-known, but the computational cost is exorbitant, and numerical instability near singular attitudes is surprisingly troublesome.

Therefore, vnoide implements **IK based on analytical solutions** for typical serial-linked legs and arms.



kinematics

Inverse kinematics of the leg

I want to find the joint angle that allows the foot to be in a specific position and posture, using the hip joint as a reference point.

Generally, it cannot be solved by hand calculation,

The case where the three axes of the hip joint and the two axes of the ankle joint intersect at one point is relatively easy to calculate.

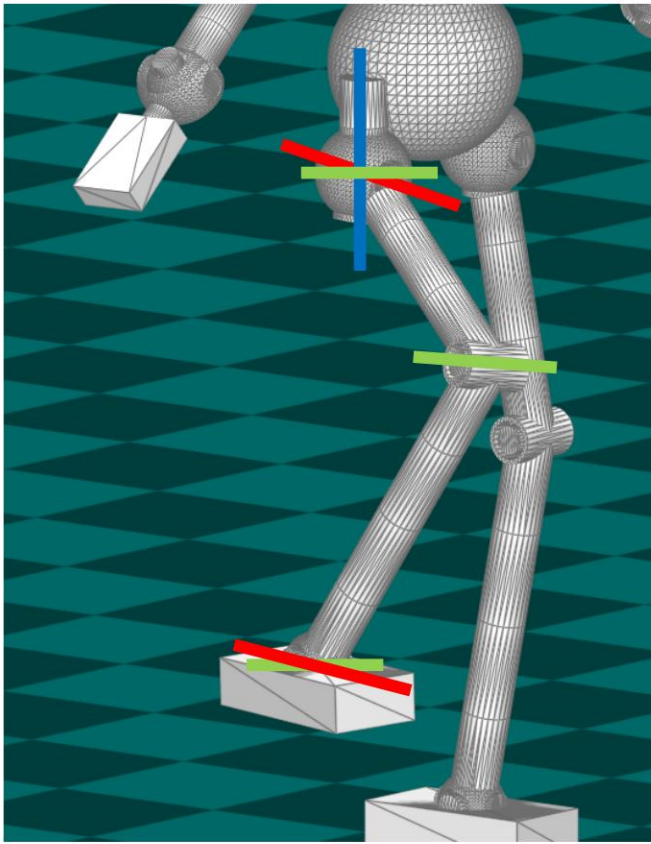
• Determine the knee flexion angle based on the distance between the foot and the base of the hip joint.

• Determine the two axes of the ankle based on the relative position of the foot
and the base of the hip joint.

• Determine the three axes of the hip joint based on the relative rotation
between the foot and the base of the hip joint.

* Implemented in IkSolver::CompLegIk

*For detailed derivation, please refer to [vnoid/doc/vnoid.pdf](#)



The way the joint axes are positioned differs slightly from one humanoid robot to another, so let's investigate.

vnoid sample robot

HRP-5P, ASIMO

Yaw-Roll-Pitch-Pitch-Roll

Kawasaki RHP Friends

Yaw-Roll-Pitch-Pitch-Roll-Pitch

Unitree G1

Pitch-Roll-Yaw-Pitch-Pitch-Roll

* The axes do not intersect at a single point.

kinematics

Inverse kinematics of the arm

Determine the joint angle such that the hand is in a predetermined position and posture, using the shoulder as a reference point.

Since there are 7 axes, one axis is redundant. It's slightly more troublesome than legs.

• The elbow yaw angle is specified from the outside.

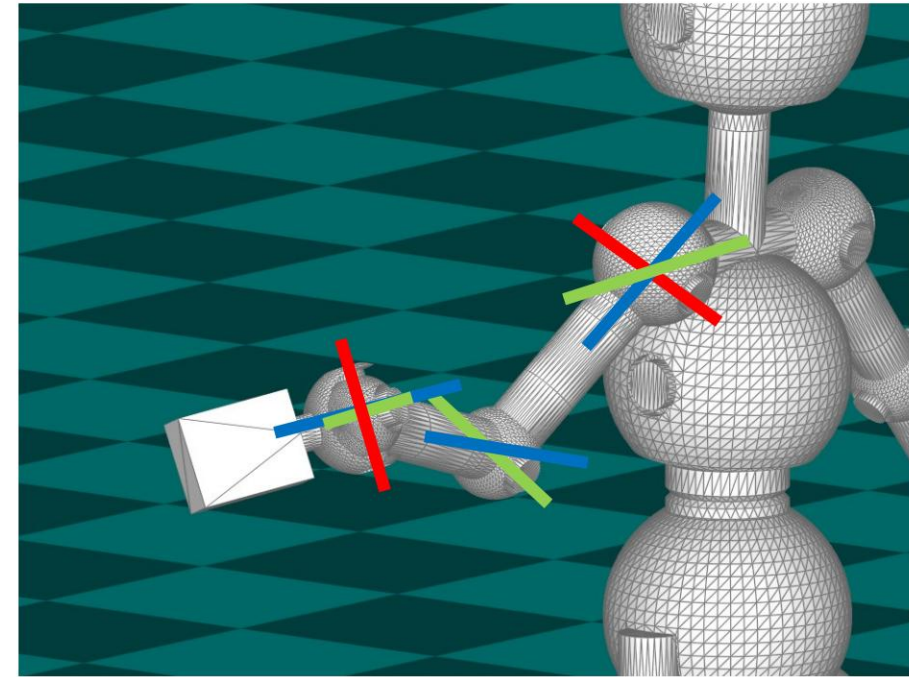
• Determine the elbow flexion angle based on the distance between the wrist and the base of the shoulder.

• Determine the three axes of the shoulder based on the relative position of the wrist
and the base of the shoulder.

• Determine the three axes of the wrist based on the relative rotation between the
wrist and the shoulder base.

*For detailed derivation, please refer to [vnoide/doc/vnoide.pdf](#)

* Implemented in `IkSolver::CompArmlk`



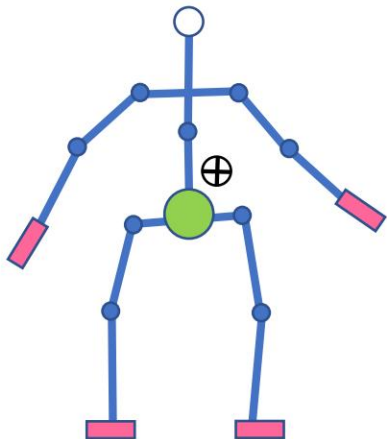
kinematics

Inverse kinematics of the center of mass

From the position of the center of gravity, the position and rotation of the limbs, we can determine the position and rotation of the base link and the angles of each joint.

Problems specific to humanoid (or more broadly, floating-based) systems: This is crucial because it forms the foundation of the stabilization controller!

Since it cannot be solved by manual calculation, it is calculated using iterative calculations with IK of the arms and legs.



To bring the center of gravity to the desired position while the limbs are fixed, the position of the base link should be adjusted.

* Assume that the joints of the torso and neck are fixed.

Pseudocode for center of gravity IK *Implemented in IkSolver::Complk

Initialize the base link position to an appropriate position (such as the center of gravity).

Repeat several times (about 10 times):

Calculate joint angles using inverse kinematics of the arms and legs.

Calculating the center of gravity position using forward kinematics (center of gravity FK)

The base link position is adjusted by the difference between the FK position and the IK position.

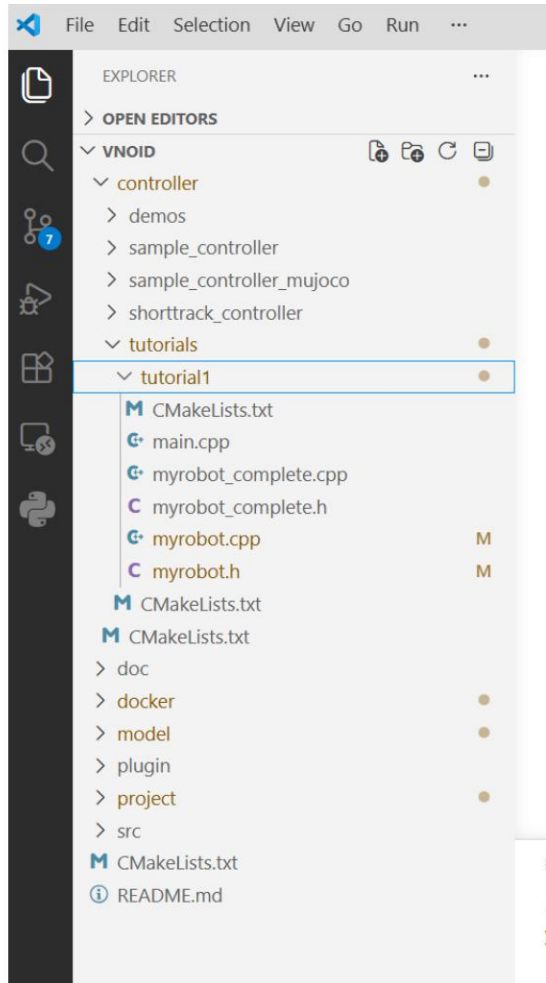
end Iteration

It converges sufficiently after about 10 iterations. The net computation time is only a few tens of microseconds (very quick!).

kinematics

Open the source code used in the tutorial.

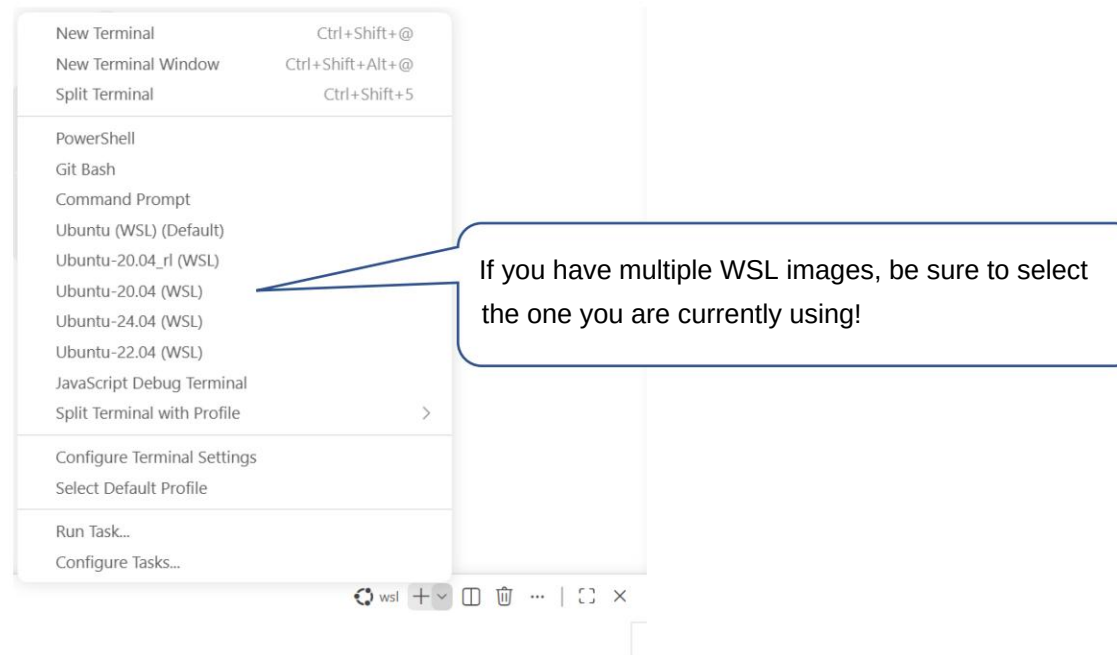
Open `vnoird/controller/tutorials/tutorial1/myrobot.cpp` from the VS Code Explorer view.



Having a terminal open in VS Code is convenient because you can complete your work entirely within VS Code.

Menu bar: "Terminal" → "New Terminal"

Click the icon in the upper right corner of the terminal and select WSL.



kinematics

Let's try moving the joints of a robot that's floating in mid-air.

Enable the mode in `MyRobot::Init` that allows you to directly control the robot's base link.

```
base_actuation = true;
```

Set the base link position in `MyRobot::Control`. This will fix the robot in mid-air.

```
base.pos_ref = Vector3(0.0, 0.0, 1.0);
```

Try setting the joint angle command values for the robot and making it assume your desired pose:

Spread your arms to the sides.

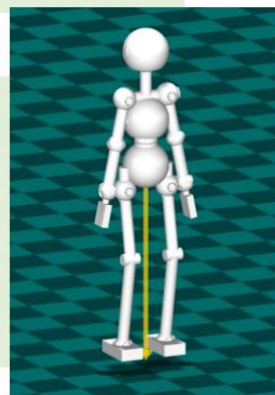
```
joint[ 5].q_ref = -0.2;
joint[12].q_ref = 0.2;
```

Bend your knees slightly.

```
joint[20].q_ref = -0.2;
joint[21].q_ref = 0.4;
joint[22].q_ref = -0.2;
```

```
joint[26].q_ref = -0.2;
joint[27].q_ref = 0.4;
joint[28].q_ref = -0.2;
```

Save with
Ctrl+S. Make it a habit to save every time you edit your code.



If you copy and paste code from a PDF:
Open in a browser, not a reader.

Rebuild (make) in the terminal and run Choreonoid again. Since selecting the project via the GUI every time is troublesome, it's better to specify the project file directly from the command line.

```
bin/choreonoid ../ext/vnoid/project/tutorials/tutorial1/vnoid_tutorial1_project.cnoid
```

Use a gamepad to control arm movements.

```
joystick.readCurrentState();
joint[ 5].q_ref = -0.5 - 0.5*joystick.getPosition(Joystick::L_STICK_H_AXIS); joint[12].q_ref = 0.5 + 0.5*joystick.getPosition(Joystick::R_STICK_H_AXIS);
```

*For details on gamepads, please refer to the official manual : <https://choreonoid.org/ja/manuals/latest/tips/gamepad/supported-gamepads.html>

Accepting input from USB devices in the WSL environment is cumbersome, so I'll use the Choreonoid virtual gamepad feature.



Supports left and right analog sticks

kinematics

Let's try moving it using IK this time.

Add fksolver and iksolver to the member variables of MyRobot.

```
...
#include "fksolver.h"
#include "iksolver.h"

class MyRobot : public Robot{ public:

    ...
    FkSolver      fk_solver;
    IkSolver      ik_solver;
```

Add the following code to MyRobot::Control()

```
Robot::Sense(timer, base, foot, joint);
...
base.pos_ref = Vector3(0.0, 0.0, 1.0);

foot[0].pos_ref = Vector3(0.0, -0.1, 0.4); foot[1].pos_ref
= Vector3(0.0, 0.1, 0.4);

ik_solver.Comp(param, base, hand, foot, joint, false, true);
...
Robot::Actuate(timer, base, joint);
```

Base target position

Foot target position

IK calculation

Let's try it with center of gravity IK this time.

```
centroid.com_pos_ref = Vector3(0.0, 0.0, 1.0);

foot[0].pos_ref = Vector3(0.0, -0.1, 0.4); foot[1].pos_ref
= Vector3(0.0, 0.1, 0.4);

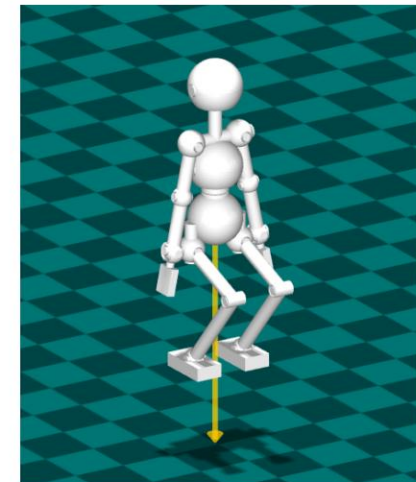
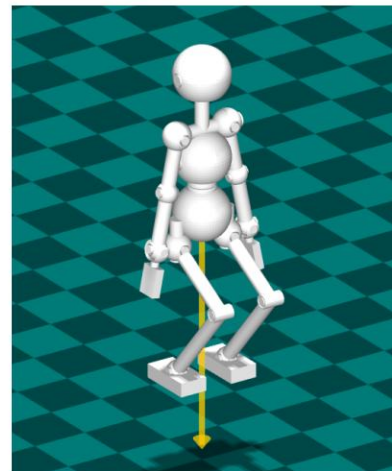
ik_solver.Comp(&fk_solver, param, centroid, base, hand, foot, joint, false, true);
```

Target position of the center of gravity

IK calculation (center of gravity IK version)

While they don't look much different, the base link moves slightly when you swing your arms, indicating that the center of gravity is calculated to be in a specific position.

It should look something like this.



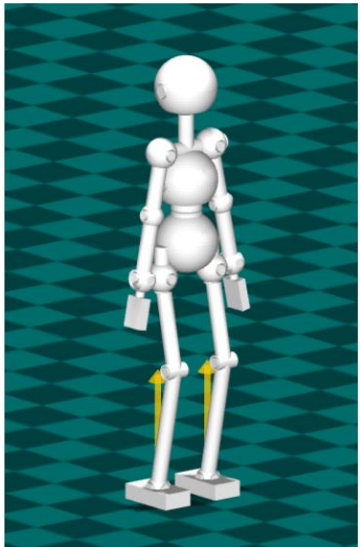
kinematics

Let's try putting the robot down on the floor.

Disable the option to fix the base link in mid-air within MyRobot::Init.

```
base_actuation = false;
```

If you run it again... the robot lands.



Try bending your knees in that position.

The gamepad input should be used to determine the target height of the center of gravity.

```
centroid.com_pos_ref.y() = joystick.getPosition(L_STICK_H_AXIS);  
centroid.com_pos_ref.z() = joystick.getPosition(L_STICK_V_AXIS);
```

Let's try moving our center of gravity up, down, left, and right to see how long it takes to fall over.

Center of gravity dynamics

Centroidal Dynamics

- Step 2: 30 minutes

- Theory

- Center of gravity dynamics

- Balance control

- Practical Application

- Try enabling the stabilizer.

- It shouldn't tip over even if you move it quickly.

- Try tilting the bass

Center of gravity dynamics

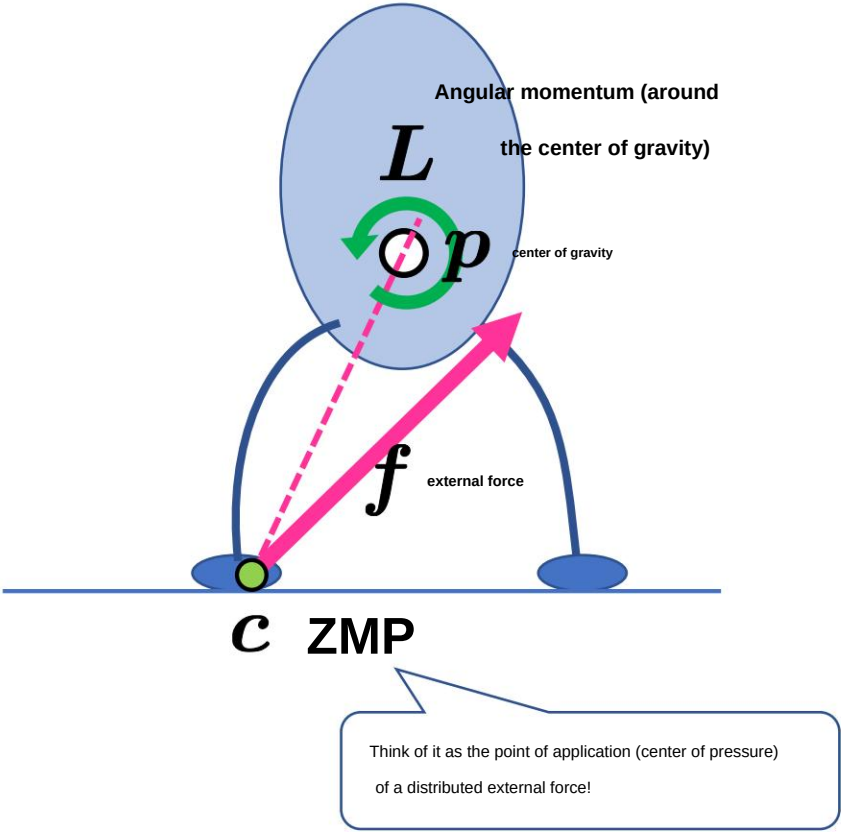
Balance control for humanoid robots is...

First, the center of gravity; second, angular momentum!

What is center of gravity dynamics?

Dynamics describes the relationship between a robot's momentum (translational momentum, angular momentum) and external forces (translational force, moment) !

It's convenient because you can consider it without having to think about internal information such as joint angles and joint torques!



$$m\ddot{\mathbf{p}} = \mathbf{f} - m\mathbf{g}$$
$$\dot{\mathbf{L}} = (\mathbf{c} - \mathbf{p}) \times \mathbf{f}$$

* While this relationship generally holds true even with contact with surfaces other than the floor, for simplicity, we will only consider contact with the floor here.

*Generally, the yaw moment around the ZMP also acts, but for simplicity, we will ignore it.

Center of gravity dynamics

Assuming the ground reaction force is directed towards the center of gravity:

$$\mathbf{f} = \frac{m}{T^2}(\mathbf{p} - \mathbf{c})$$

T is a suitable proportionality constant (it doesn't necessarily have to be a constant in general, but it's usually treated as a constant for ease of handling).

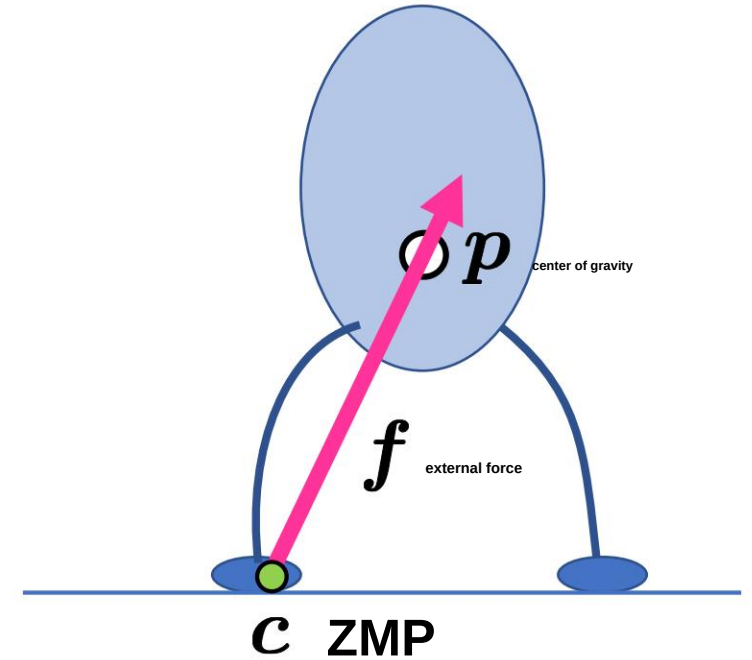
Substitute

$$\ddot{\mathbf{p}} = \frac{1}{T^2}(\mathbf{p} - \mathbf{c}) - \mathbf{g}$$

Linear Inverted Pendulum Mode (LIPM) (the three-dimensional version)!

Because the moment around the center of gravity becomes zero.

$$\dot{\mathbf{L}} = \mathbf{0}$$



LIPM is well-known, but it's important to have a proper understanding of what it means.

LIPM is not a physical law that always holds true between the center of mass and ZMP.

When the center of gravity and ZMP follow LIPM, the angular momentum (its roll-pitch components) remains unchanged.

In other words, the angular momentum changes by the amount of deviation from the LIPM.

Center of gravity dynamics

Furthermore, if we keep the height of the center of gravity from the floor constant,

$$p_z - c_z \equiv h$$

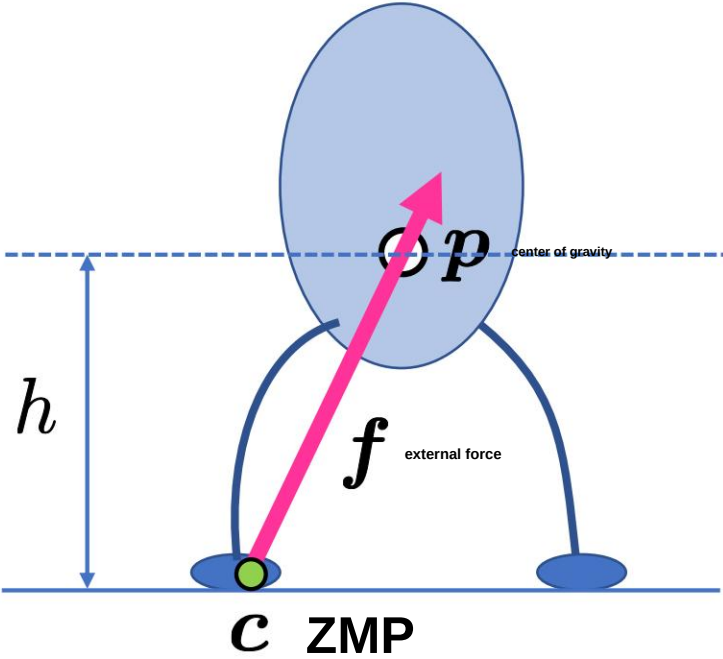
$$\ddot{p}_z = \frac{1}{T^2} h - g = 0 \quad \text{twist} \quad T = \sqrt{\frac{h}{g}}$$

Linear Inverted Pendulum Mode (LIPM) (the two-dimensional version)!

$$\ddot{p}_x = \frac{1}{T^2} (p_x - c_x)$$

$$\ddot{p}_y = \frac{1}{T^2} (p_y - c_y)$$

This is comes up when you read old papers.



Center of gravity dynamics

LIPM can be decomposed into two equations.

$$\ddot{\mathbf{p}} = \frac{1}{T^2}(\mathbf{p} - \mathbf{c}) - \mathbf{g}$$



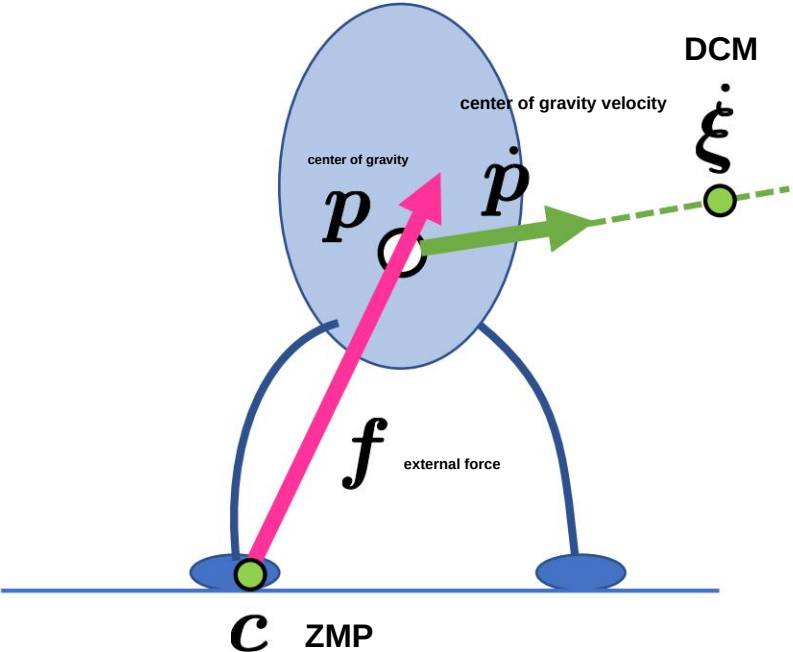
Disassembly!

$$\dot{\mathbf{p}} = -\frac{1}{T}(\mathbf{p} - \boldsymbol{\xi})$$

$\boldsymbol{\xi} = \mathbf{p} + T\dot{\mathbf{p}}$ This is called DCM (Divergent Component of Motion).

Since it's a stable system, the center of gravity will remain Following DCM

$$\dot{\boldsymbol{\xi}} = \frac{1}{T}(\boldsymbol{\xi} - \mathbf{c}) - T\mathbf{g}$$



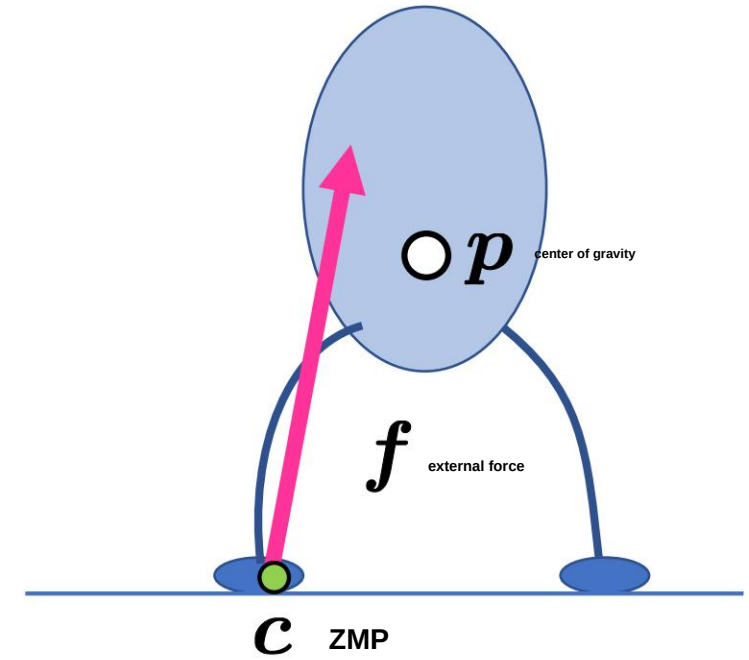
Center of gravity dynamics

What happens if the ground reaction force is not directed towards the center of gravity?

$$\mathbf{f} = \frac{m}{T^2}(\mathbf{p} - \mathbf{c}) + m\boldsymbol{\delta}$$

Component directed toward the center of gravity

Center of gravity
acceleration offset



Substitute

$$\ddot{\mathbf{p}} = \frac{1}{T^2}(\mathbf{p} - \mathbf{c}) - \mathbf{g} + \boldsymbol{\delta}$$

Expressed in terms of DCM dynamics:

$$\dot{\boldsymbol{\xi}} = \frac{1}{T}(\boldsymbol{\xi} - \mathbf{c}) - T\mathbf{g} + T\boldsymbol{\delta}$$

Center of gravity dynamics

What happens if the ground reaction force is not directed towards the center of gravity?

The ground reaction force generates a moment equal to the displacement from the center of gravity, thus changing the angular momentum.

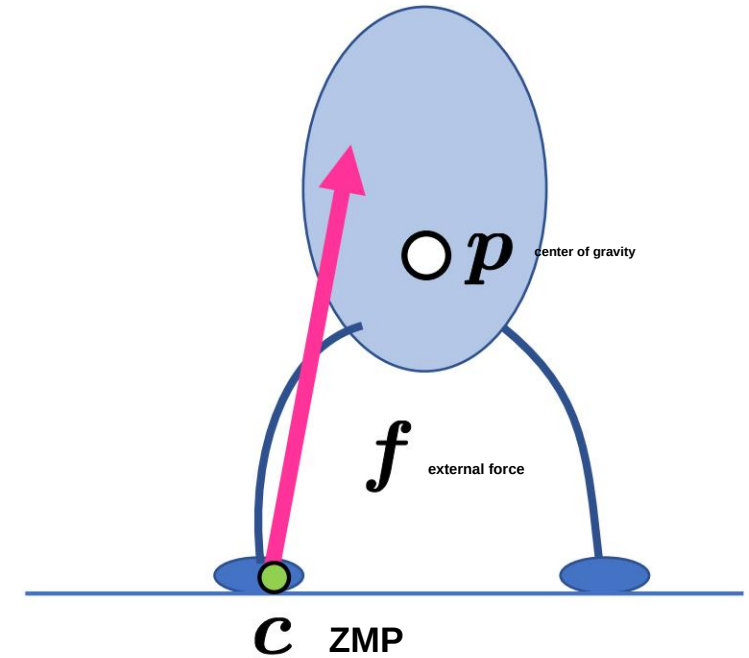
$$\dot{\mathbf{L}} = -m(\mathbf{p} - \mathbf{c}) \times \boldsymbol{\delta}$$

If we keep the floor height of the center of gravity constant and focus only on the roll and pitch directions,

$$\dot{L}_x = mh \delta_y$$

$$\dot{L}_y = -mh \delta_x$$

Angular momentum can be controlled by utilizing the additional acceleration of the center of mass (relative to LIPM)!



*This is essentially the same as what is discussed in other literature using Centroidal Moment Pivot (CMP), but while CMP is proposed as an extension of ZMP, it is more appropriate to consider it here as "additional acceleration of the center of gravity," so we will proceed in that way.

Center of gravity dynamics

Let's consider a balance control law that applies the principles we've discussed so far.

• Determine the moment required to stabilize the base link horizontally using the PD control law.

$$\dot{L}^{\text{ref}} = -(K_P \theta^{\text{sense}} + K_D \omega^{\text{sense}})$$

• Determine the additional acceleration of the center of gravity required to apply the moment.

$$\delta_x^{\text{ref}} = -\frac{\dot{L}_y^{\text{ref}}}{mh} \quad \delta_y^{\text{ref}} = \frac{\dot{L}_x^{\text{ref}}}{mh}$$

• ZMP feedback rule for making DCM command values track target values

$$c^{\text{ref}} = c^{\text{target}} + K_{\text{DCM}}(\xi^{\text{ref}} - \xi^{\text{target}})$$

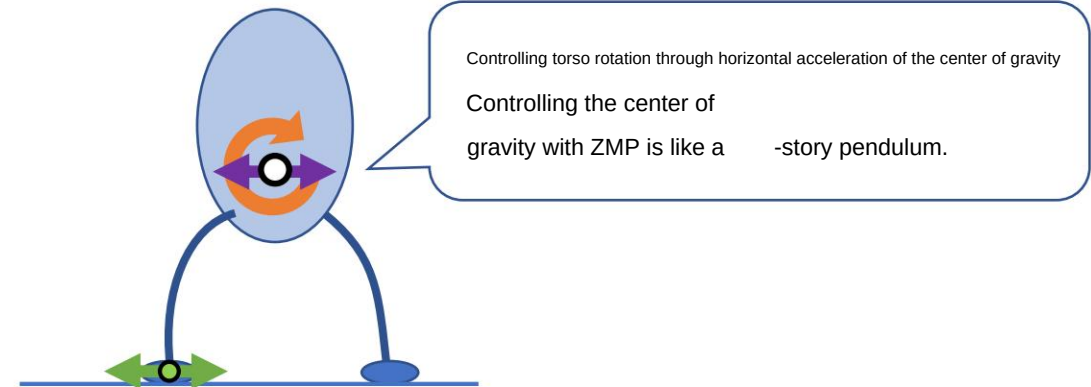
• Update the DCM command value (and also update the center of gravity position command value).

$$\dot{\xi}^{\text{ref}} = \frac{1}{T}(\xi^{\text{ref}} - c^{\text{ref}}) - Tg + T\delta^{\text{ref}}$$

* Implemented in Stabilizer::Update

Correspondence with parameter names within vvoid

K_P	orientation_ctrl_gain_p
K_D	orientation_ctrl_gain_d
K_{DCM}	dcm_ctrl_gain



$$\dot{p}^{\text{ref}} = -\frac{1}{T}(p^{\text{ref}} - \xi^{\text{ref}})$$

Center of gravity dynamics

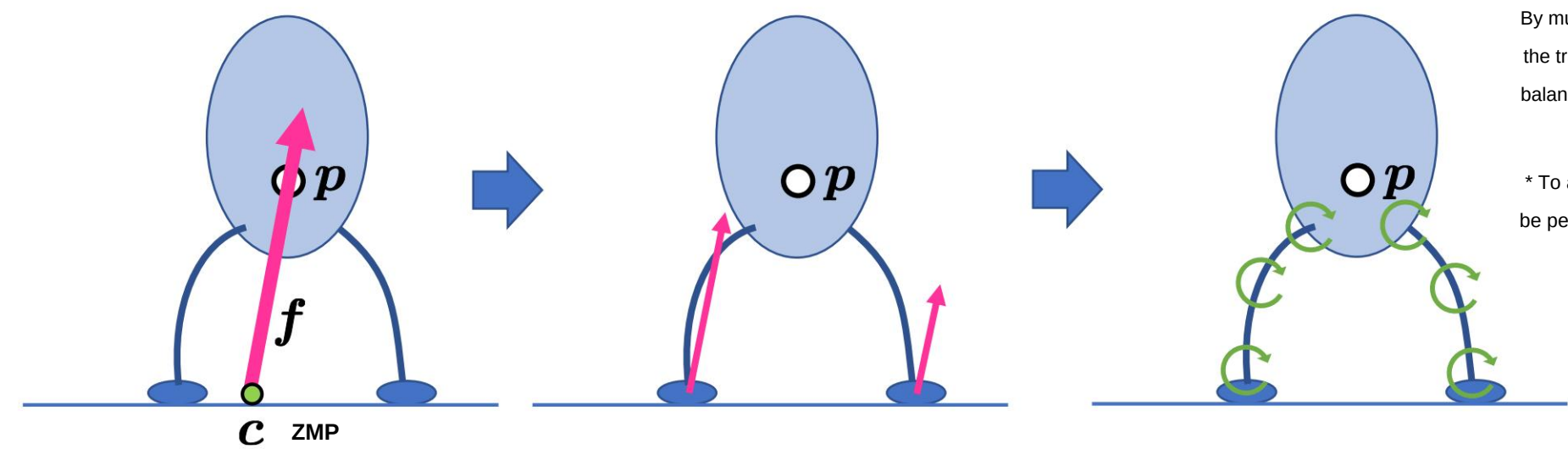
What happens to the ZMP command values after that?

• Determine the ground reaction force command values for each foot from the ZMP command values.

* Implemented in Stabilizer::CalcForceDistribution

• Determine joint torque command values from ground reaction force

command values *This is done within IkSolver::Complk



By multiplying the ground reaction force command value by the transpose of the Jacobian matrix of the legs, the statically balanced joint torque can be calculated.

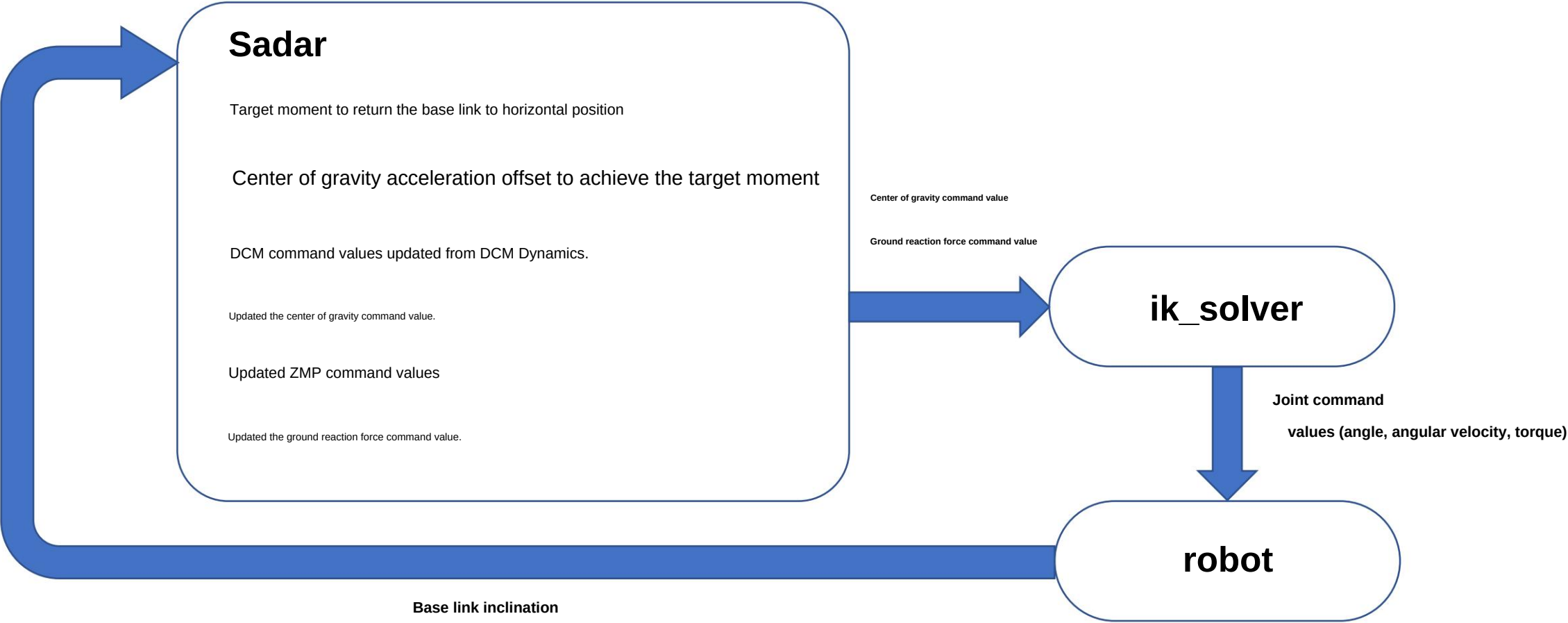
* To account for inertia, inverse dynamics calculations should be performed, but this is not yet implemented.

*We do not use ground reaction force control via feedback from ground reaction force sensors.

*Please note that the effectiveness of this design will vary depending on the simulation environment and the characteristics of the actual machine, so consider this merely one example.

Center of gravity dynamics

Balance control law using only a gyroscope



Center of gravity dynamics

Let's try enabling the stabilizer.

Add `stabilizer` to the member variables of MyRobot.

```
...
#include "stabilizer.h"

class MyRobot : public Robot{ public:

    ...
    Stabilizer      r;
```

Configure the stabilizer in MyRobot::Init. *You don't need to worry about the meaning of each parameter for now.

```
stabilizer.orientation_ctrl_gain_p = 50.0;
stabilizer.orientation_ctrl_gain_d = 5.0;
stabilizer.dcm_ctrl_gain = 2.0;
```

Make the ankle joint more flexible (reduce PD gain)

```
joint[22].Set(100.0, 20.0, 100.0);
joint[23].Set(100.0, 20.0, 100.0);
joint[28].Set(100.0, 20.0, 100.0);
joint[29].Set(100.0, 20.0, 100.0);
```

Set the relevant parameters within MyRobot::Init.

```
centroid.com_pos_ref = Vector3(0.0, 0.0, param.com_height); centroid.dcm_ref
= Vector3(0.0, 0.0, param.com_height); centroid.dcm_target = Vector3(0.0,
0.0, param.com_height); foot[0].pos_ref = Vector3(0.0, -0.10, 0.0);
foot[1].pos_ref = Vector3(0.0, 0.10, 0.0); foot[0].contact_ref
= true; foot[1].contact_ref = true;
```

The following should be done inside MyRobot::Control:

```
void MyRobot::Control(){
    Robot::Sense(timer, base, foot, joint);

    fk_solver.Comp(param, joint, base, centroid, hand, foot);

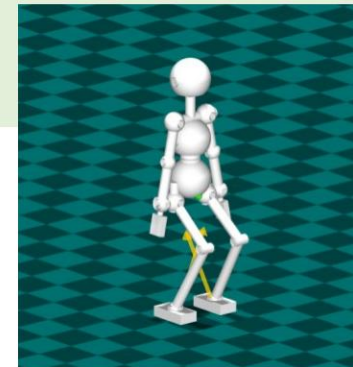
    ...

    stabilizer.Update(timer, param, centroid, base, foot);

    ik_solver.Comp(&fk_solver, param, centroid, base, hand, foot, joint, false, true);

    Robot::Actuate(timer, base, joint);

    timer.Countup();
}
```



If you run it again...

Center of gravity dynamics

Let's try enabling the stabilizer.

Try controlling the DCM target value with a gamepad.

```
joystick.readCurrentState();  
centroid.dcm_target.z() = param.com_height + 0.1*joystick.getPosition(Joystick::L_STICK_V_AXIS);
```

The stabilizer updates the center of gravity position command value internally, so you need to input the DCM target value instead.

You can also move the DCM target value forward, backward, left, and right, and manipulate the rotation command value of the base link, so give it a try.

Let's see how the response changes when we vary the control gain.

stabilizer.orientation_ctrl_gain_p

stabilizer.orientation_ctrl_gain_d

To determine the amount of restoring moment with respect to the base link inclination PD Gain

stabilizer.dcm_ctrl_gain

The feedback gain applied to ZMP to bring the DCM command value closer to the DCM target value. *Note that theoretically, it must be > 1 for the stabilization effect to work.

Center of gravity dynamics

Let's try enabling the visualizer.

Add `visualizer` to the member variables of MyRobot.
Add the `Visualize` function for visualization.

```
...
#include "visualizer.h"

class MyRobot : public Robot{ public:

    ...
    Visualizer      visualization;

    ...
    void Visualize();
}
```

Configure the visualizer in MyRobot::Init. Be careful not to enter too large a value, as this may cause the shared memory size to become insufficient.

```
visualizer.header.numMaxFrames      = 10000; =
visualizer.header.numMaxLines      0;
visualizer.header.numMaxSpheres = 10; visualizer.header.numMaxBoxes
= 10; visualizer.header.numMaxCylinders = 0;
visualizer.header.numMaxLineVertices = 0; visualizer.Open();
```

Maximum number of frames

Maximum number of
shapes per frame

Call Visualize from within MyRobot::Control.

```
void MyRobot::Control(){
    Robot::Sense(timer, base, foot, joint);
    ...
    Robot::Actuate(timer, base, joint);

    Visualize();

    timer.Countup();
}
```

Obtain a rendering frame based on the current time (visualized at 40 FPS in this example).

Define the Visualize function as follows:

```
void MyRobot::Visualize(){ int iframe
= (int)(timer.time/0.025); Visualizer::Frame* frame =
visualizer.data->GetFrame(iframe); if(!frame) return;

frame->time = timer.time;

Visualizer::Sphere* sphere = visualizer.data->GetSphere(iframe, 0); sphere->pos sphere->radius
= 0.02f; sphere-> = centroid.com_pos_ref;
>color = Eigen::Vector3f(0.0f,
1.0f, 0.0f); sphere->alpha = 0.5f;

sphere = visualizer.data->GetSphere(iframe, 1); sphere->pos
sphere->radius = centroid.zmp_ref;
= 0.02f; sphere->color =
Eigen::Vector3f(1.0f, 0.0f, 0.0f); sphere->alpha = 0.5f;

}
```

The center of gravity command value is visualized as a green sphere, and the ZMP command value is visualized as a red sphere.

If you run it again...

Walking pattern generation and walking control

Walking Pattern Generation & Walking Control

- Step 3: 30 minutes
 - Theory
 - DCM-based gait pattern generation method
 - Landing position and timing correction
 - Practical Application
 - Try moving forward using a walking pattern.
 - Try controlling it with a joystick
 - Create a walking pattern using a program and try making the program walk.
 - Let them go up the stairs

Walking pattern generation

- What is a walking pattern? The target trajectory of the landing position and center of gravity when walking on two legs.

No matter how well the balance control system works, it cannot properly track a target trajectory that has been arbitrarily created.

We need to create a target trajectory that is physically sound.

- How to create a walking pattern based on center of gravity dynamics

Target trajectory for angular momentum

The roll-pitch components of angular momentum are assumed to be constant (effectively identically zero).

$$\dot{L}_x = \dot{L}_y = 0$$

Yaw component (rotational component) of angular momentum

$$\dot{L}_z = \eta_z$$

Assume that the moment acting from the floor to the feet does not exceed the static friction limit.

Target trajectory of the center of gravity

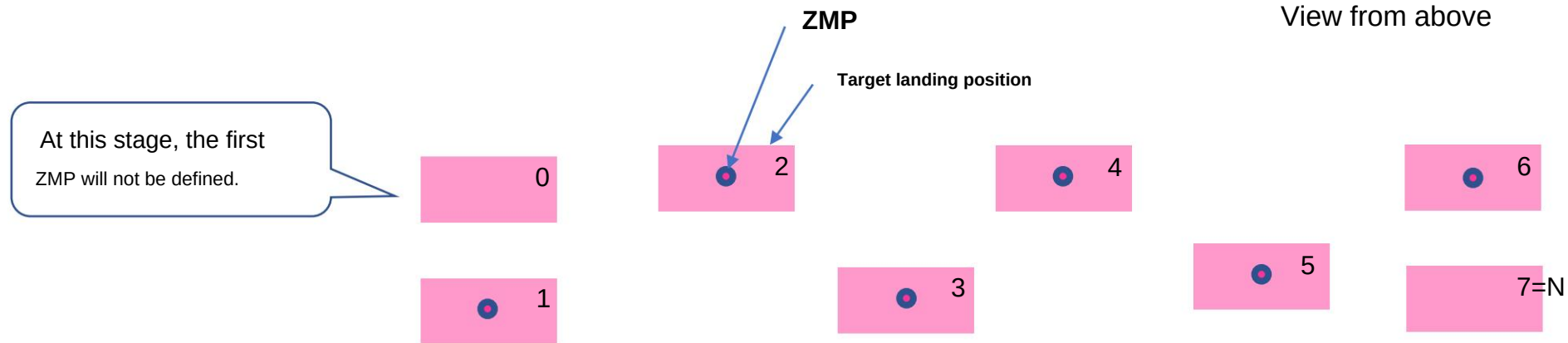
Generated using the DCM equation

$$\dot{\xi} = \frac{1}{T}(\xi - c) - Tg$$

Walking pattern generation

Steps for generating walking patterns

1. Determine the desired footstep and ZMP sequence.

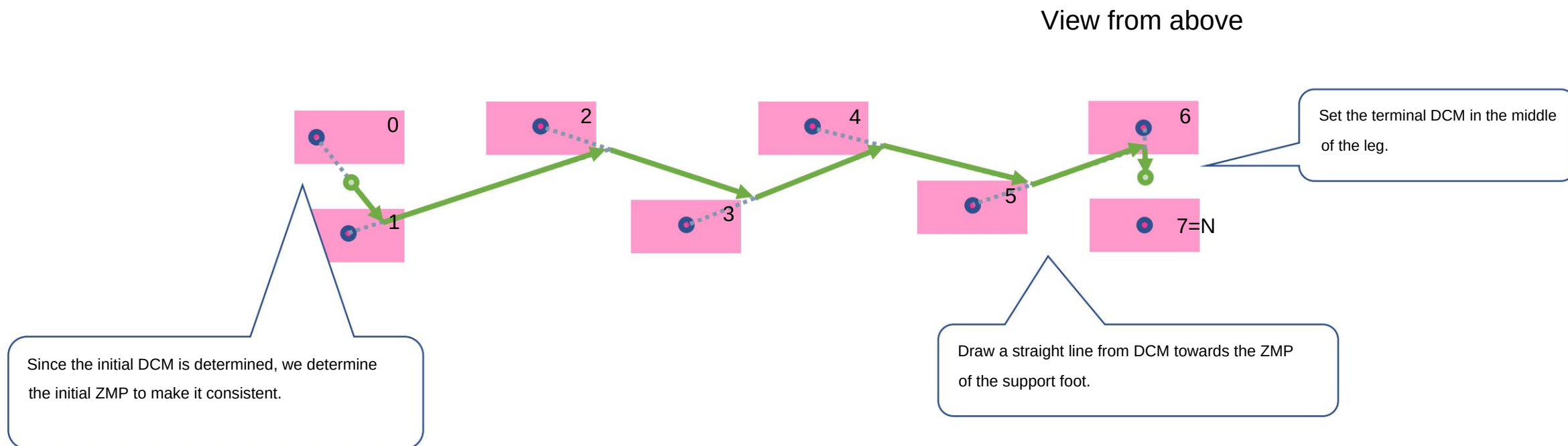


* Implemented in FootstepPlanner::Plan

*For simplicity, we will assume there is no double-foot support phase, and that the ZMP does not move during the single-foot support phase (this can be extended, but the calculations become more complicated).

Walking pattern generation

2. Draw the DCM's trajectory backward.



* Implemented in FootstepPlanner::GenerateDCM

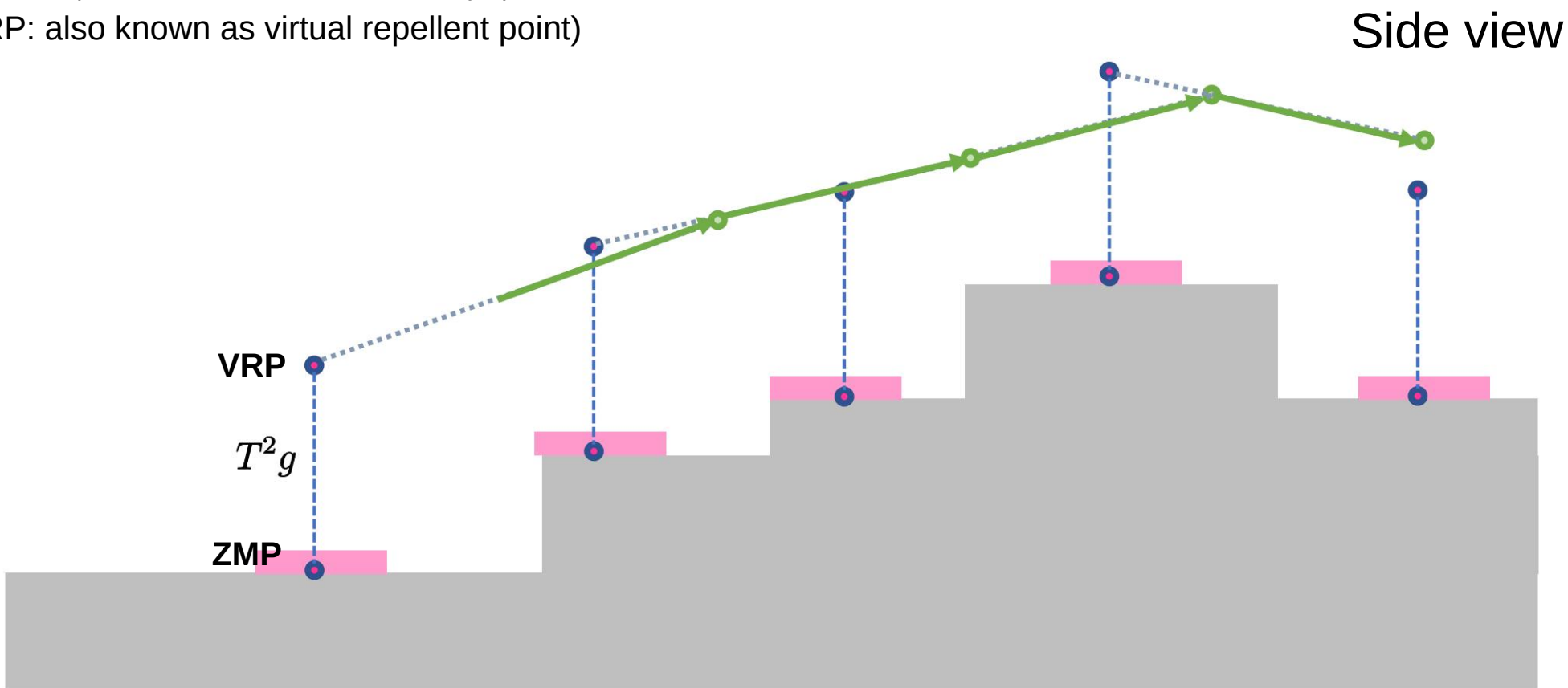
Walking pattern generation

2. Draw the DCM's trajectory backward.

The method is the same even for patterns that include vertical movement,

Consider the point where ZMP is shifted vertically upward.

(VRP: also known as virtual repellent point)



Landing position and timing correction

While walking is possible using only the aforementioned balance control, the range of motion of the ZMP is limited to within the support area, so it can only withstand small disturbances.

A mechanism that adjusts the next landing position and timing in accordance with the movement of the DCM command value is extremely effective.

The concept of correcting landing timing

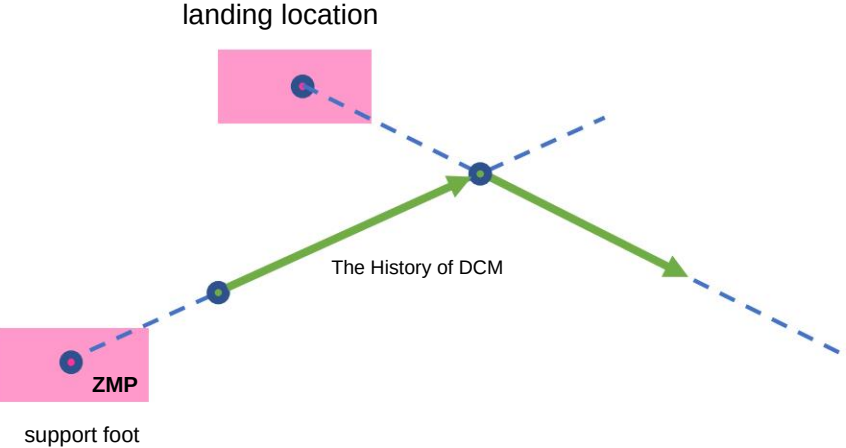
The timing is measured by "how far the DCM is from the ZMP".

The concept of landing position correction

The landing position is determined by the "relative position of the landing foot and the DCM at the time of landing (DCM offset)".

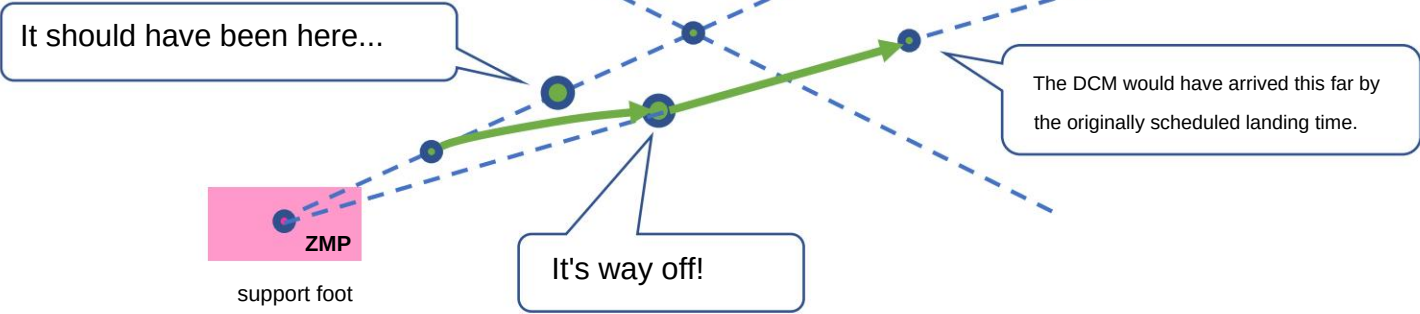
Landing position and timing correction

Initial planned

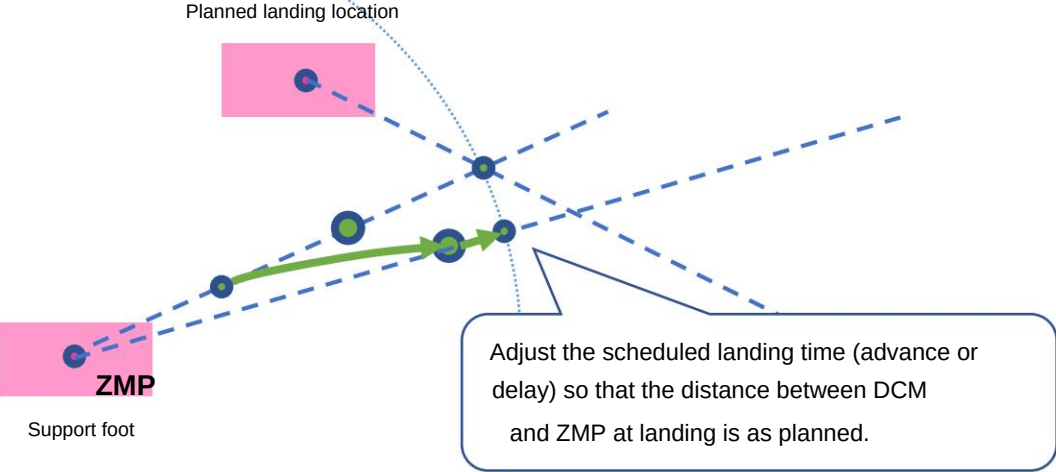


The DCM was delayed due to external disturbances.

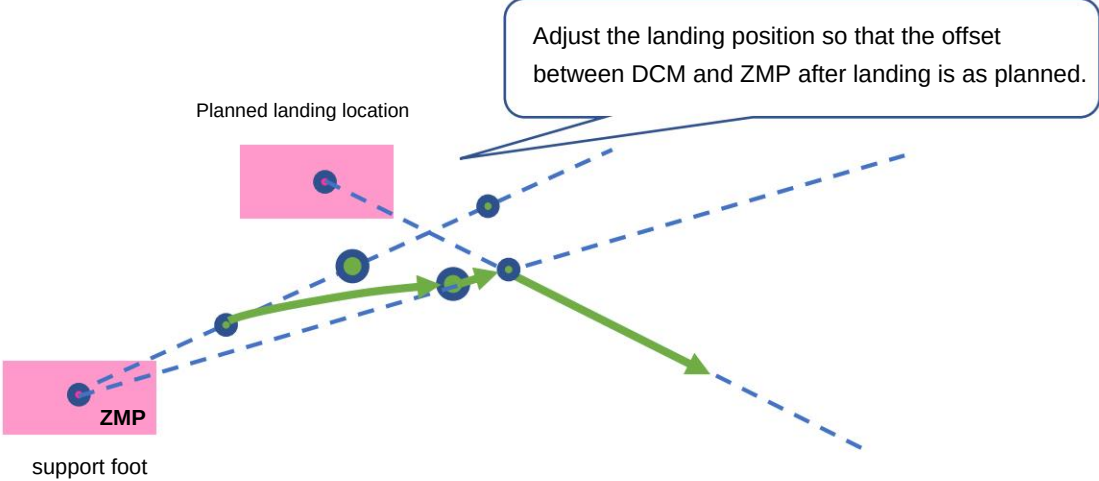
~View from above~



Timing adjustment

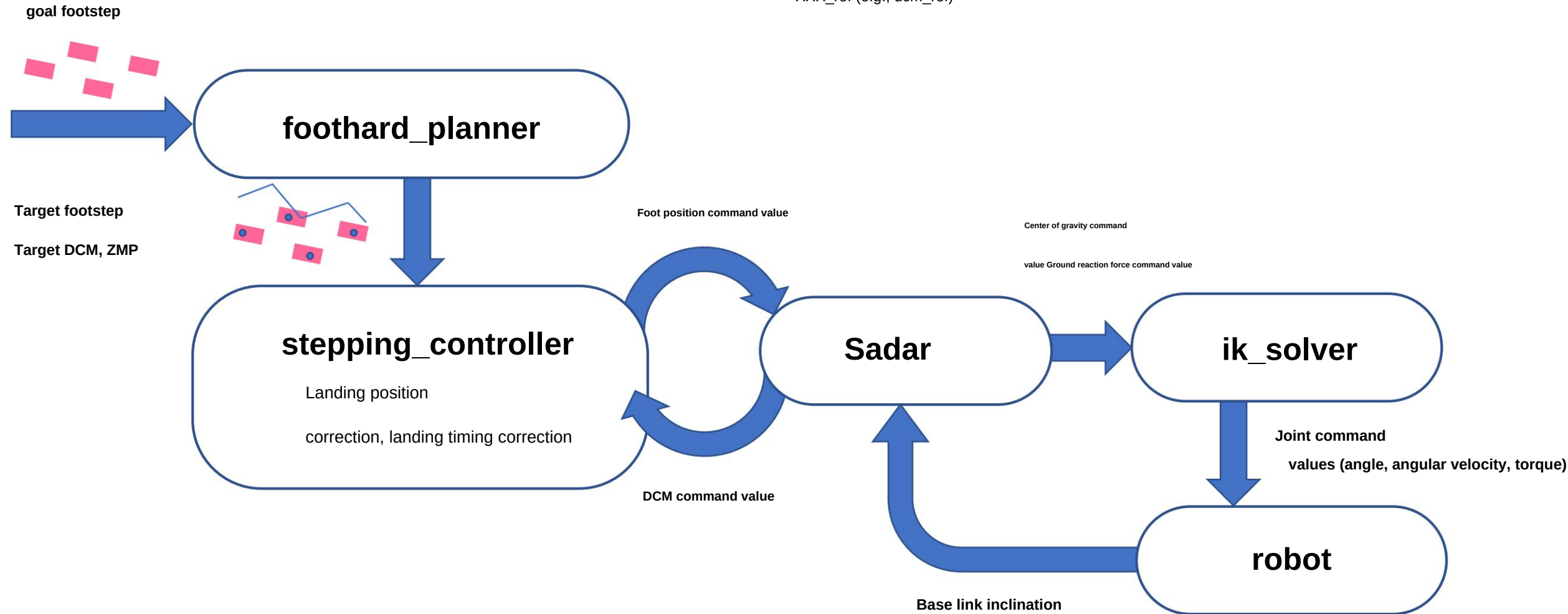


Landing position adjustment



Walking pattern generation

The role of each module in vvoid



Be careful, as Vnoid's internal variable naming policy is a bit complicated.

XXX_target (e.g., dcm_target)

$\ddot{y}$ Planned target value $\ddot{y}$

XXX_ref (e.g., dcm_ref)

Modified command value for control

Walking pattern generation

Let's try adding a walking pattern generator.

Add footstep_planner and stepping_controller to the member variables of MyRobot.

```
...  
#include "footstep_planner.h" #include  
"stepping_controller.h"  
  
class MyRobot : public Robot{ public:  
  
    ...  
    Footstep          footstep;  
    Footstep footstep_buffer;  
    FootstepPlanner footstep_planner;  
    SteppingController stepping_controller;
```

Setting the stepping_controller within MyRobot::Init

```
stepping_controller.swing_height = 0.05;  
stepping_controller.dsp_duration = 0.05;
```

Height to lift your leg

Length of the double-foot support phase

Walking pattern generation

Generate a footstep to take a few steps straight ahead.

```

Step step;
step.duration = 0.5;
step.spacing = 0.2;

step.stride = 0.0;
footstep.steps.push_back(step);

step.stride = 0.1;
footstep.steps.push_back(step);
footstep.steps.push_back(step);
footstep.steps.push_back(step);
footstep.steps.push_back(step);

step.stride = 0.0;
footstep.steps.push_back(step);
footstep.steps.push_back(step);
footstep.steps.push_back(step);

footstep.steps[0].foot_pos[0] = foot[0].pos_ref;
footstep.steps[0].foot_pos[1] = foot[1].pos_ref; footstep.steps[0].dcm
= centroid.dcm_ref;

footstep_planner.Plan(param, footstep);
footstep_planner.GenerateDCM(param, footstep);

footstep_buffer.steps.push_back(footstep.steps[0]);
footstep_buffer.steps.push_back(footstep.steps[1]);
  
```

Set common parameters (landing cycle and distance between left and right feet)

To shift weight before stepping forward, take a step with a stride length of zero. In this case, do not march in place, only shift your center of gravity.

You can take 5 steps with a stride length of 0.1m.

To stop, take a few steps with zero stride length.

As initial conditions, set the position command values for the left and right feet and the DCM command value.

Have the planner calculate additional information.

Copy the beginning to the buffer.

Walking pattern generation

Let's try adding a walking pattern generator.

The following should be done inside MyRobot::Control:

```
void MyRobot::Control(){
    Robot::Sense(timer, base, foot, joint);

    fk_solver.Comp(param, joint, base, centroid, hand, foot);

    stepping_controller.Update(timer, param, footstep, footstep_buffer, centroid, base, foot);

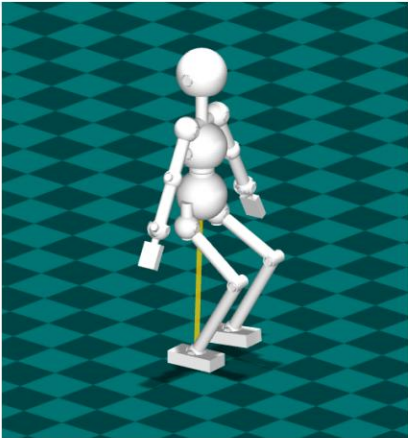
    stabilizer.Update(timer, param, centroid, base, foot);

    ik_solver.Comp(&fk_solver, param, centroid, base, hand, foot, joint, false, true);

    Robot::Actuate(timer, base, joint);

    timer.Countup();
}
```

If you run it again...



I should be able to walk!

Let's try it.

Try changing the gait parameters and see how your walking style changes.

stride	Stride length (amount of movement for forward and backward)
sway	Lateral movement
view	Vertical movement
turn	Turning amount
S	Distance between left and right feet
Theatrical releases are a major part of the process.	landing period

It's dangerous to start walking before you've even landed, so try waiting a few seconds before you start walking.

The elapsed time can be obtained from timer.time.

Walking pattern generation

Let's try to make it controllable with a gamepad.

Add the following code to MyRobot::Control
(before stepping_controller)

```
while(footstep.steps.size() > 2)
    footstep.steps.pop_back();

Step step;
step.spacing = 0.2;
step.stride = -0.2*joystick.getPosition(Joystick::L_STICK_V_AXIS); =
               -0.5*joystick.getPosition(Joystick::L_STICK_H_AXIS); step.turn
footstep.steps.push_back(step);
footstep.steps.push_back(step);
footstep.steps.push_back(step);

footstep_planner.Plan(param, footstep);
footstep_planner.GenerateDCM(param, footstep);
```

Delete all steps except the one closest to you.

Extend foot steps to reflect
gamepad input

If you run it again...

Walking pattern generation

Switch to an environment model with stairs and correct the initial posture.

Open the following project file:

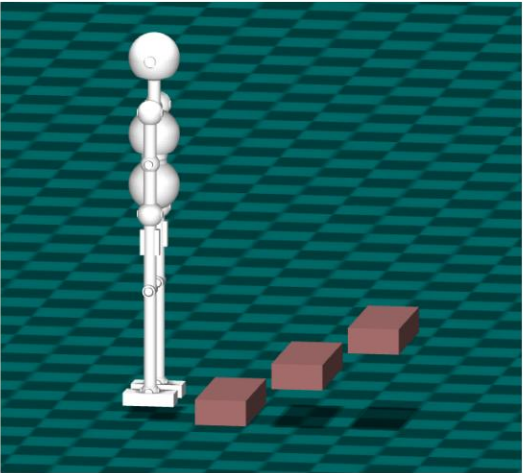
Choreonoid/ext/vnoid/project/tutorials/tutorial1/vnoid_tutorial1_project.cnoid

Change the floor model to one with stairs.

```
-
  id: 7
  name: "Floor" plugin:
  Body class:
  BodyItem is_checked:
  true data:

  The mechanism is the same as for foot steps that move in the z direction.
  ...
```

When I tried loading it...



Correct the robot's initial posture
(because dropping it from too high a height will cause its position to shift...).

```
-
  id: 3
  name: "Unitychan" plugin:
  Body class: BodyItem
  is_checked: true data:
  file: "${SHARE}/model/

  sample_robot/sample_robot_ver2.body" format: CHOREONOID-BODY currentBaseLink: "HIPS"
  rootPosition: [ 0, 0, 0.85 ] rootAttitude:
  [ 1, 0, 0, 0, 1, 0, 0, 0, 1 ]
  jointDisplacements: [

  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
  0 ] initialRootPosition: [ 0, 0, 0.85 ]
  initialRootAttitude: [ 1, 0, 0, 0, 1, 0, 0, 0, 1 ]

  ...
```

Corrected the initial height of the base link (in two places).

Walking pattern generation

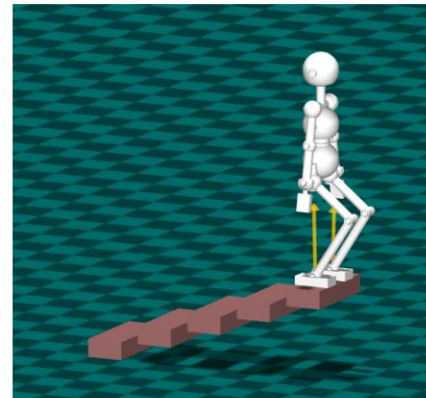
Let's adjust our footsteps to make them look like we're climbing stairs.

```
step step;  
step.duration = 0.7;  
step.spacing = 0.2; step.stride  
= 0.0;  
footstep.steps.push_back(step); step.stride  
= 0.3; step.climb = 0.1;  
footstep.steps.push_back(step);  
footstep.steps.push_back(step);  
footstep.steps.push_back(step);  
footstep.steps.push_back(step);  
footstep.steps.push_back(step); step.stride  
= 0.0; step.climb = 0.0;  
  
footstep.steps.push_back(step);  
footstep.steps.push_back(step);  
footstep.steps.push_back(step);
```

The parameters of stepping_controller were also corrected.

```
stepping_controller.swing_height = 0.15;  
stepping_controller.dsp_duration = 0.10;
```

Will I be able to go? ...



In conclusion

- How to create a walking controller using Choreonoid and vno3d

- We hope this will help you understand the principles of model-based gait control.
 - I want you to know that simply walking can be done without reinforcement learning or model-based predictive control.

- Limitations of model-based

approaches • Whole-body movement, running, jumping, human-like simulations, getting up, etc. • How to combine with data-driven methods?

- Simulation Challenge

- With the information presented in the tutorial and a little ingenuity, you can complete the lower difficulty levels of the athletic course. Reduce your time. Various strategies are needed to improve.
- High difficulty courses are a bit tough.

- vno3d has many applications.

- There will be a short presentation on vno3d applications in our lab.

- vno3d's gait control laws are perfectly applicable to actual robots.

- Our laboratory has experience implementing this in small bipedal robots.

Reference information

•Kinesiology

Since this is basic content, it's difficult to provide specific reference information, but... Professor Kajita's

"Humanoid Robot" with 6-axis IK Choreonoid and HRP4CHandler

•Center of Gravity

Dynamics: For information on the balance control laws and landing corrections implemented in vno3d, please refer to the following.

Y. Tazaki, "DCM Modulation: A Three-Axis Rotation Stabilization Technique for Bipedal Locomotion Control," 2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Hangzhou, China, 2025, pp. 4735-4742. *The yaw control proposed in the paper is not yet implemented in vno3d's stabilizer.

The method for landing correction is almost the same as below.

M. Khadiv, A. Herzog, SAA Moosavian and L. Righetti, "Walking Control Based on Step Timing Adaptation," in IEEE Transactions on Robotics, vol. 36, no. 3, pp. 629-643, June 2020, doi: 10.1109/TRO.2020.2982584.

•Walking pattern generation

The method implemented in vno3d is the one proposed by Englsberger. (J. Englsberger, C. Ott and A. Albu-Schäffer, "Three-Dimensional Bipedal Walking Control Based on Divergent Component of Motion," in IEEE Transactions on Robotics, vol. 31, no. 2, pp. 355-368)

Professor Murooka compares various methods for generating walking patterns and controlling tracking.

<https://github.com/isri-aist/BaselineWalkingController>

Reference information

Challenge page (this year's version is under preparation): <https://ytazz.github.io/vnoid/>

Past Tutorials

Humanoids2022 Tutorial

<https://ytazz.github.io/vnoid/humanoids2022tutorial.html>

2023 Technical Training Session

<https://ytazz.github.io/vnoid/hvac2023tutorial.html>

2024 Technical Training Session

<https://connpass.com/event/321220/>

<https://connpass.com/event/321221/>

2025 Spring Meeting

<https://connpass.com/event/345666/>

Tutorial recording

<https://www.youtube.com/channel/UCG-P4LJu1PmewkuBrn4saRw>

vnoid tutorial blog

<https://koomiy.github.io/>